

指针

复习

迷宫

- 1.游戏概述
- 2.游戏地图
- 3.方向控制
- 4.胜利条件
- 5.关卡升级

目录

- | | |
|-------------|----|
| 1. 指针概述 | 了解 |
| 2. 指针定义与初始化 | 熟悉 |
| 3. 指针和数组 | 重点 |
| 4. 二级指针 | 熟悉 |
| 5. 指针和函数 | 重点 |
| 6. 指针与字符串 | 重点 |

指针概述

指针，表示的就是**内存的地址**，所谓指针变量，也就是保存了内存地址的变量。

指针也称为定位符或指示符！

计算机中所有的数据都必须放在内存中，为了正确地访问这些数据，必须为每个字节都编上号码，就像网上买东西一样，需要提供地址，快递才能精准送达，程序中也是根据编号准确地找到某个字节。

将内存中字节的编号称为**地址（Address）或指针（Pointer）**。

指针定义

使用 指针 来保存一个 变量 的地址，在 C++ 中定义指针变量需要在变量名和类型之间加一个 *，要获取一个变量的地址，需要在变量前面加上 &。

定义指针变量语法

```
type *varName;
```

定义了一个指针变量 varName，其类型为 type*，其中 type 可以是任意的数据类型

指针定义

获取变量地址

语法

&varName

使用 & 符号获取一个变量的地址。

获取指针变量指向的值

语法

***varName**

使用 * 获取一个指针变量所指向的地址的值，也称指针的间接引用。

指针初始化

在定义指针的时候，可以直接初始化，不过指针变量只能初始化为变量的地址，不可以直接初始化为值。

语法

```
type *pName = &varName;
```

定义了一个指针变量 pName，其类型为 type，它指向了变量 varName。

指针定义与初始化 案例

```
int num = 100; //定义一个整型变量num
```

```
int *p = &num; //定义个指针变量指向num变量的地址
```

```
cout << "num = " << num << " &num = " << &num << endl;
```

```
cout << "*p = " << *p << " p = " << p << endl;
```

指针运算++ --

指针变量保存的是变量的地址，**变量的地址本质上也是一个整数**，因此，可以使用运算符对指针变量进行运算。

指针变量常用的运算符就是加法和减法运算符，其中，自增 和 自减运算符用的最多。对指针变量进行自增或者自减也叫做指针变量的移动。

指针运算++ --

定义一个整型变量num，在定义一个指针变量指向num，试着把指针变量自增自减。

```
int num = 100 ;  
int* p = &num;  
cout << "p:" << p << endl;  
p++;  
cout << "p:" << p << endl;  
p--;  
cout << "p:" << p << endl;
```

指针和数组

数组 是用来保存同一种 类型 的一组数据的集合，同时，数组中的所有元素在内存中是连续存放的，每个元素的地址在内存中是连续的。

在 C++ 中，可以将数组名看成是一个 指针，它指向数组的第 1 个元素，也就是索引为 0 的元素，在 C++ 中，我们将第 1 个元素的地址称为数组的首地址。

注意，我们可以将数组名看成是指针，但其实它们并不完全等价，因为数组名其实是一个 常量，不可以被修改，而指针是可以移动的，即是一个变量。

指针和数组

数组名可以看成指针

```
int arr[3] = {1, 3, 4};  
for(int i = 0; i < 3; i++){  
    cout << "arr[" << i << "] = " << arr[i] << endl;  
}
```

```
int arr[3] = {1, 3, 4};  
for(int i = 0; i < 3; i++){  
    cout << "arr[" << i << "] = " << *(arr+i) << endl;  
}
```

在 `*(arr+i)` 这个表达式中，`arr` 是数组名，指向数组的第 0 个元素，表示数组首地址，`arr+i` 指向数组的第 `i` 个元素，`*(arr+i)` 表示取第 `i` 个元素的数据，它等价于 `arr[i]`

指针和数组

使用指针指向数组名

语法

```
type *指针名 = 数组名 ;
```

```
int arr[3] = {1, 2, 3};
```

```
int *p = arr;
```

定义了一个指针，其指向了数组名，直接可以通过指针的移动来访问数组中的元素。

指针和数组

```
int arr[3] = {1, 2, 3};
```

```
int *p = arr;
```

```
for(int i = 0; i < 3; i++){  
    cout << "p[" << i << "] = " << *(p+i) << endl;  
}
```

```
for(int i = 0; i < 3; i++){  
    cout << "p[" << i << "] = " << *p++ << endl;  
}
```

二级指针

将指针指向一个普通的数据类型，比如 int 类型、double 类型等，那么这个指针就是一级指针，即保存的是变量的地址。

同时，除了可以将指针指向普通的数据类型，还可以定义一个指针指向另一个指针，这个就是**指针的指针，也就是二级指针**。

语法

```
int num = 100;
```

```
int *pnum = &num;
```

```
int **ppn = &pnum;
```

二级指针

```
int num = 100;  
int *pnum = &num;  
int **ppn = &pnum;  
cout << *pnum << endl;  
cout << **ppn << endl;
```



输出一样吗？

多级指针

定义多级指针，即指针的级数没有限制

```
int a = 99;
```

```
int *pa1 = &a;
```

```
int **pa2 = &pa1;
```

```
int ***pa3 = &pa2;
```

```
int ****pa4 = &pa3;
```

```
int *****pa5 = &pa4;
```

```
int ******pa6 = &pa5;
```

指针和函数

函数的参数不仅可以是基本的数据类型，还可以是指针类型，用指针变量作函数参数可以将函数外部的地址传递到函数内部，使得在函数内部可以操作函数外部的数据，并且这些数据不会随着函数的结束而被销毁。

如果，需要在函数内部修改传入的基本类型的参数的值，那么我们只能通过指针的方式来传递参数。有点像引用一样。

指针作为函数参数

语法

```
returnType funcName(paramType1 *param1)
{
    return val
}
```

指针作为函数参数

```
void func1 ( int v1 ) {  
    v1++;  
}  
int main(){  
    int num = 100;  
    cout << num << endl;  
    func1(num);  
    cout << num << endl;  
    return 0;  
}
```

```
void func1 ( int *v1 ) {  
    (*v1)++;  
}  
int main(){  
    int num = 100;  
    cout << num << endl;  
    func1(&num);  
    cout << num << endl;  
    return 0;  
}
```

指针作为函数参数 案例

利用指针作为参数交换两个变量的值。

指针作为函数参数 案例

利用指针作为参数交换两个变量的值。

```
void swap(int *num1, int *num2)
{
    int temp;
    temp = *num1;
    *num1 = *num2;
    *num2 = temp;
}
```

指针作为函数返回值

函数的返回值定义为指针类型，用来返回一个变量的地址，但不能返回 **局部变量的地址**。

用指针作为函数返回值时需要注意的一点是，函数运行结束后会销毁在它内部定义的所有局部数据，包括局部变量、局部数组和形式参数，**函数返回的指针不要指向这些数据**。

指针作为函数返回值

语法

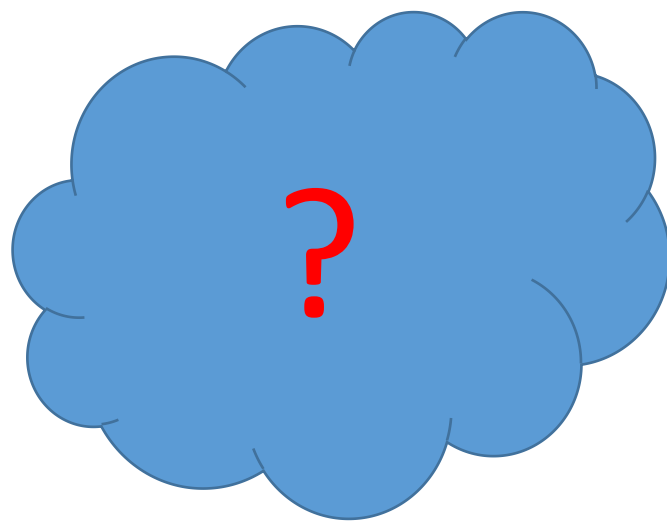
```
type * funcName()  
{  
    return val;  
}
```

指针作为函数返回值

```
int * func1(int *num){
    *num += 100;
    return num;
}
int main(){
    int v1 = 1;
    cout << v1 << endl;
    int *p = func1(&v1);
    cout << v1 << endl;
    cout << *p << endl;
    return 0;
}
```

指针作为函数返回值

```
int * func2(){  
    int num = 100;  
    return &num;  
}  
int main(){  
    int * p = func2();  
    cout << *p << endl;  
    return 0;  
}
```



指针与字符串

字符串 是可以通过 字符数组 来实现的，因此，数组指针 的相关操作其实同样适用于字符串。

同时，还可以直接通过指针的方式来定义一个字符串。

语法1

```
char *varName = "str";
```

语法2

```
char *varName;  
varName = "str";
```

指针与字符串

```
char * pc1 = "hello world";
```

```
char *pc2;
```

```
pc2 = "this is C++";
```

指针与字符数组

通过指针，来遍历字符串

```
char str[] = "hello world";  
int len = strlen(str);  
for(int i = 0; i < len; i++)  
{  
    cout << *(str+i);  
}
```

```
char str[] = "hello world";  
int len = strlen(str);  
char* pstr = str;  
for(int i = 0; i < len; i++)  
{  
    cout << *pstr++;  
}
```

NULL指针和void指针

一个指针不指向任何数据，称之为空指针，用 **NULL** 表示。注意，NULL 是区分大小写的，即 NULL 不能写成 null。

在定义指针时或者再指针使用完毕，不再使用时，一定推荐将指针设置为 NULL，表明该指针不再指向任何数据。

定义

```
#define NULL ((void *)0)
```

(void *)0 表示把数值 0 强制转换为 void * 类型，最外层的 () 把宏定义的内容括起来，在进行宏定义时，推荐这么做，防止发生歧义。

NULL指针和void指针

定义NULL指针

在定义指针变量时，

```
int *p;
```

不建议这样写，一般如果指针定义出来不指向任何地址是比较危险的，叫做野指针。

建议写出下面形式

```
int *p = NULL ;
```

NULL指针和void指针

在c++98，字面常量0的类型既可以是整型，也可以是一个无类型指针（void *）。

在某些场景（例如，函数重载）的使用会遇到麻烦，C++11引入关键字**nullptr**来避免二义性，nullptr是一个“指针空值常量”，仅可以被隐式转换为指针类型。

所以以后指针初始化赋值就使用nullptr.

```
int *p = nullptr;
```

NULL指针和void指针

void 的意思就是 “无类型”，void 指针则为 “无类型指针”，void 指针可以指向任何类型的数据。所以 void 指针一般被称为通用指针或者泛指针，也可以叫做万能指针。

定义

`void *p`

使用 `void *` 定义了一个万能指针。

NULL指针和void指针

void指针使用

- 1.在 C++ 中在任何时候都可以用 void 类型的指针来代替其他类型的指针，void 指针可以指向任何数据类型的变量。
- 2.如果要通过 void 指针去获取它所指向的变量值时候，需要先将 void 指针强制类型转换成和变量名类型相匹配的数据类型指针后再进行操作。
- 3.任何类型的指针都可以赋值给 void 指针，无需进行强制类型转换。

NULL指针和void指针

void 指针可以指向任何数据

```
int num= 100;
```

```
void *p = &num;
```

```
cout << "p = " << p << endl;
```

```
//cout << "*p = " << *p << endl; //void 指针不可以直接使用，必须经过类型转换后才可以使用
```

```
cout << " *p = " << *((int*)p) << endl; // 将 void 指针强制类型转换成了 int 指针
```

指针与引用的区别

指针的本质是一个变量的地址，而引用的本质是变量的别名。同时，定义指针值可以不初始化，并且可以定义空指针，但定义引用时，必须初始化，并且不能定义空引用。

每一种编程语言都使用指针，不止 C 语言和 C++ 使用指针。只是 C++ 将指针暴露给了用户(程序员)，而 Java 和 C# 等语言则将指针隐藏起来了。

指针与引用的区别

区别

- 1.从现象上看，指针在运行时可以改变其所指向的值，而引用一旦和某个对象绑定后就不再改变。
- 2.从内存分配上看，程序为指针变量分配内存区域，而不为引用分配内存区域，因为引用声明时必须初始化，从而指向一个已经存在的对象。引用不能指向空值。
- 3.从编译上看，程序在编译时分别将指针和引用添加到符号表上，符号表上记录的是变量名及变量所对应地址。指针变量在符号表上对应的地址值为指针变量的地址值，而引用在符号表上对应的地址值为引用对象的地址值。符号表生成后就不会再改，因此指针可以改变指向的对象（指针变量中的值可以改），而引用对象不能改。这是使用指针不安全而使用引用安全的主要原因。从某种意义上来说引用可以被认为是不能改变的指针。

不存在指向空值的引用这个事实，意味着使用引用的代码效率比使用指针的要高。因为在使用引用之前不需要测试它的合法性。相反，指针则应该总是被测试，防止其为空。

理论上，对于指针的级数没有限制，但是引用只能是一级。

指针常量与常量指针

1.const修饰常量 即指针常量

```
int num = 100 , num2=200;
```

```
int * const p= &num;
```

```
//p = &num2 ; //错误
```

```
*p = 111 ; //正确
```

由于const修饰的是p，所以p不能修改，即指针的值不能修改，即指针的指向不能修改

指针常量与常量指针

2.const修饰指针 即常量指针

```
int num = 100 , num2=200;
```

```
const int * p= &num;
```

```
p = &num2 ; //正确
```

```
//*p = 111 ; //错误
```

由于const修饰的是*p，所以*p不能修改

指针常量与常量指针

3.const修饰指针 又常量

```
int num = 100 , num2=200;
```

```
const int * const p= &num;
```

```
//p = &num2 ; //错误
```

```
//*p = 222 ; //错误
```

都不能修改

引用和常量引用

引用的本质其实是：指针常量

即：`int * const p = #`

特点就是：和指针常量的特点一样，指向的值能改变，指针的指向不能改变。

常量引用的本质：`const int * const p = #`

特点就：指针的指向和指向的值都不能更改。

练习 题目

函数原型：

```
void checkMax(int *p1,int *p2,int *pmax);
```

实现找出两个数中最大值。

练习 代码

```
void checkMax(int *p1,int *p2,int *pmax){  
    if(*p1 > *p2){  
        *pmax = *p1;  
    }else{  
        *pmax = *p2;  
    }  
}
```

```
int main(){  
    int n1,n2,n3;  
    cin >> n1 >> n2;  
    checkMax(&n1,&n2,&n3);  
    cout << n3 << endl;  
}
```

知识点总结

- | | |
|------------------|----|
| 1. 指针概述 | 了解 |
| 2. 指针定义与初始化 | 熟悉 |
| 3. 指针和数组 | 重点 |
| 4. 二级指针 | 熟悉 |
| 5. 指针和函数 | 重点 |
| 6. 指针与字符串 | 重点 |
| 7. NULL指针和void指针 | 熟悉 |
| 8. 指针与引用的区别 | 重点 |

作业

面试题

引用与指针相同点和不同点？

下节课预告

复合数据类型 结构体